```
import numpy as np

# 1. Activation function (Sigmoid)
def sigmoid(x):
    return 1 / (1 + np.exp(-x))

# 2. Derivative of sigmoid
def sigmoid_derivative(x):
    return x * (1 - x)

# 3. Input dataset (XOR problem)
X = np.array([[0,0],
              [0,1],
              [1,0],
              [1,1]])

# Output dataset
y = np.array([[0],
              [1],
              [1],
              [0]])

# 4. Initialize weights randomly
np.random.seed(1)
input_neurons = 2
hidden_neurons = 2
output_neurons = 1

# Weights
W1 = np.random.rand(input_neurons, hidden_neurons)
W2 = np.random.rand(hidden_neurons, output_neurons)

# Bias
b1 = np.random.rand(1, hidden_neurons)
b2 = np.random.rand(1, output_neurons)
```

```
# Learning rate
lr = 0.1

# 5. Training using Backpropagation
for epoch in range(10000):

    # Forward propagation
    hidden_input = np.dot(X, W1) + b1
    hidden_output = sigmoid(hidden_input)

    final_input = np.dot(hidden_output, W2) + b2
    final_output = sigmoid(final_input)

    # Error
    error = y - final_output

    # Backpropagation
    d_output = error * sigmoid_derivative(final_output)

    error_hidden = d_output.dot(W2.T)
    d_hidden = error_hidden * sigmoid_derivative(hidden_output)

    # Update weights and bias
    W2 += hidden_output.T.dot(d_output) * lr
    W1 += X.T.dot(d_hidden) * lr

    b2 += np.sum(d_output, axis=0, keepdims=True) * lr
    b1 += np.sum(d_hidden, axis=0, keepdims=True) * lr

# 6. Testing
print("Final Output after Training:")
print(final_output)
```

```
Final Output after Training:
[[0.07304485]
 [0.93084242]
 [0.93122512]
 [0.07564609]]
```